

Triaging Model

Input patient data(Name, Age, Gender, Symptoms)

Measure patient's Vital Signs(GCS, SBP, DBP, MAP, Pulse, Temp, RR, Fio2)

Measurements

#Use of a list to hold multiple vitals signs in order would make the data easily readable and easier to maintain and easier to change. Each vital sign is stored in the patient's record(usually a data structure) for later use and processing

#All measurements taken must be handled by a clinical physician or nurse. These measures can only be taken physically.

Measure Heart Rate

Measure(Pulse)

Measure GCS

Measure(GCS)

Measure SBP

Measure(SBP)

Measure DBP

Measure(DBP)

Measure MAP

Measure(MAP)

Measure Temp

Measure(Temp)

Measure Respiratory Rate

Measure(RR)

Measure Fio2

Measure(Fio2)

#Usually the data received from patients can have gaps such as erroneous, different data types and missing data

#Therefore the data has to go through processing to make it clean and ready for processing in order to create an error free diagnosis.

#At this point, the data should reside in the data structure.

Steps in Data Cleaning using python

Step 1:

Determining the actual values in the actual column. So using the `.nunique()` fn we can visualize the actual number of unique values in a specific column.

Step 2:

Mapping the unique elements to the specified elements we want. Using `map_name = {}` we can achieve this.

Step 3:

Apply the map to create a new and clean column. This is done by using `.map()` function and apply the changes to make the changes we need

Step 4:

Drop the dirty column. Use the `.drop()` function.

Handling wrong data types

Step 1: Determining the actual values in the actual column. So using the `.nunique()` fn we can visualize the actual number of unique values in a specific column.

Step 2: Visualize the data. Plotting the data using built in functions in python such as `.hist()` and `.boxplot()` to study the data for outliers and physiologically impossible values. Flag physiological impossible values.

Step 3: Replace the invalid values with NaN values. Using the numpy function `np.nan`.

Step 4: Imputation of NaN values using the appropriate imputation strategy. Mean would be used if the data is normally distributed, Median if the data is skewed and Mode if only value dominates the set. Either `.median()`, `.mean()` or `.mode()` is used to get the values for imputation.

#This ensures that the data is ready to be interpreted and understood by medical professionals.

Classification

Classify(Pulse)

#We then classify the patient's pulse score with medical patterns to determine the risk

If pulse < 60

Risk_score += 3

elif pulse > 100

Risk_score +=3

#pulse < 60 is classified as Bradycardia(too slow)
#pulse > 100 is classified as Tachycardia(too fast)
#pulse is >= 60 and pulse <= 100 is classified as Normal

Classify(GCS)

#We then classify the patients's GCS score with medical patterns to determine risk.

If GCS >=9 AND <=12

Risk_score +=3

elif GCS<= 8

Risk_Score +=5

#GCS <=8 could potentially mean severe brain injury and critical condition

#GCS >=9 AND <=12 could mean moderate brain impairment

#GCS >=13 means normal consciousness

Classify(SBP)

If SBP <90

Risk_score +=5

elif SBP > 180

Risk_Score+=3

#SBP <90 means low blood pressure which may indicate shock, bleeding which could be fatal

#SBP >180 Severely high blood pressure which is an emergency

Classify(MAP)

if MAP <65

Risk_score +=5

elif MAP > 100

Risk_score +=3

#MAP <65 means poor organ perfusion

#MAP >100 means a high pressure load

Classify(Temperature)

If temperature > 39

Risk_score += 3

elif temperature <35

Risk_score += 3

#Multiple vitals can be added to the program to calculate a risk score. A risk score is used to determine how long a patient can wait before medical attention is needed.

Determine Triage Category

```
if risk_score >= 15
patient = "Critical"
elif risk_score >= 10 and risk_score <=15
patient = "High"
elif risk_score >=5 and risk_score <=10
patient = "Moderate"
else
patient = "Low"
```

#risk_score patients >= 15 need immediate treatment.

#risk_score patients >=10 need urgent treatment < 10 mins.

#risk_score patients >=5 does not need any urgent treatment and can wait.

#risk_score patients <=5 can wait.

The system should:

Display the risk score of each patient.

Display the time the patient should be treated